

Computability of the Inverse Fabius Function

**Sequential computation, effective uniform continuity,
exact inverse moduli, and certified Thue–Morse bisection**

Research report prepared for Vladimir Reshetnikov’s ProveIt project

30 August 2026

Repository snapshot: 40fdea4cc0a728189f357389e3f114a2cb00e561

Abstract

Let F be the canonical bounded Fabius function, viewed as the strictly increasing distribution function on $[0, 1]$, and let $G = F^{-1}$. We prove constructively that G is a computable real function in the Grzegorzczuk/Wikipedia sense [1, 2, 9]: it maps every computable real sequence to a computable real sequence, and it possesses a recursive modulus of uniform continuity. The same theorem holds for the repository’s totalized inverse $\overline{G} : \mathbb{R} \rightarrow [0, 1]$, obtained by clamping the argument to $[0, 1]$.

The proof is quantitative. First, the centered finite random sums give exact rational Thue–Morse splines S_N satisfying the global certificate $|S_N - F| \leq 2^{-N}$. Thus F can be evaluated at a rational point to any requested precision by a finite integer computation. Second, the Fabius density is symmetric and strictly unimodal. It follows that among all subintervals of $[0, 1]$ of a fixed length h , an endpoint interval has the least Fabius mass:

$$\min_{0 \leq x \leq 1-h} (F(x+h) - F(x)) = F(h).$$

Consequently the exact modulus of continuity of the inverse is itself:

$$\sup_{|u-v| \leq \eta} |G(u) - G(v)| = G(\min(\eta, 1)).$$

This identity reduces effective continuity to a positive lower bound for $F(h)$. A direct box event in the defining random series gives, for every $r \geq 0$,

$$F(2^{-r}) \geq \Delta_r := 2^{-r(r+1)/2} (r+1)^{-(r+1)}.$$

Hence $|u-v| < \Delta_r$ implies $|\overline{G}(u) - \overline{G}(v)| < 2^{-r}$, yielding an explicit primitive-recursive modulus. Combined with the standard flatness upper bound, this also determines the optimal binary input precision for p output bits:

$$\mu(p) = \frac{p^2}{2} + p \log_2 p + \mathcal{O}(p).$$

Finally, a tolerant bisection algorithm uses certified approximations to both $F(c)$ and the target value. It never asks an undecidable exact-equality question: when comparison is inconclusive, the inverse modulus certifies that the midpoint is already sufficiently accurate. This supplies a uniform name-transformer and proves sequential computability.

The report audits the recursive LaTeX corpus under `Analysis/FabiusFunction/docs`, separates imported repository results from the arguments proved here, relates the construction to q -binomial and sinc-product representations, Lambert- W inverse asymptotics, Legendre expansions, and Bell/Bernoulli coefficient calculi. It records both the completed post-publication Lean crosswalk for the structural inverse modulus and the remaining roadmap for the computability layer. An exact-rational Python supplement reproduces the finite-spline and modulus calculations.

Status, snapshot, and formalization boundary

Every statement labelled theorem, proposition, lemma, or corollary below is proved in this report or explicitly imported with a source pointer. The repository audit and all relative-novelty statements remain pinned to commit 40fdea4cc0a728189f357389e3f114a2cb00e561. At that snapshot, the main computability theorem, the least fixed-length Fabius-increment inequality, the exact inverse modulus, and the closed elementary modulus were absent under the formulations used here.

A post-publication Lean development now formalizes the structural inverse-modulus layer in `Analysis/FabiusFunction/Lean/FabiusFunction/InverseModulus.lean`. It proves the global unimodal shape of fixed-length Fabius increments, the endpoint lower bound, constrained forward superadditivity, local and global inverse-gap inequalities, global inverse subadditivity, attained exact moduli on both $[0, 1]$ and $\mathbb{R}$, and the exact effective-injectivity threshold. The declaration-level crosswalk is given in [section 12](#).

This update does *not* by itself formalize the closed bound Δ_r , recursive-modulus packaging, tolerant bisection, sequential computability, the combined computable-real-function theorem, or the input-precision asymptotics. Those claims retain the complete human proofs and imported-source qualifications stated below. “New” means new relative to the pinned corpus; no unqualified worldwide priority claim is made. Conjectures and research problems are explicitly labelled and are not used in the proof.

Contents

1	Main result and proof architecture	5
1.1	The theorem in one statement	5
1.2	Four proof layers	5
2	Corpus audit, conventions, and nonduplication boundary	6
2.1	Pinned repository snapshot	6
2.2	What the repository already proves	6
2.3	The gap closed here	7
2.4	Three functions that must not be conflated	7
3	Computable-analysis definitions	7
3.1	Fast names and computable sequences	7
3.2	The reciprocal uniform-continuity convention	8
4	Probability, Fourier structure, and the forward function	8
4.1	The random-series model	8
4.2	Finite sums and their exact CDFs	9
4.3	Centered finite splines and a uniform certificate	9
5	Effective positivity, strict increase, and the density shape	10
5.1	A closed positive lower bound at dyadic points	10
5.2	Strict monotonicity from the random series	11
5.3	Symmetric strict unimodality	11
6	The least-mass interval and the exact inverse modulus	12
6.1	Intervals of a prescribed geometric length	12
6.2	The inverse’s modulus is the inverse itself	13

7	Effective uniform continuity	15
7.1	A closed primitive-recursive modulus	15
7.2	The exact-dyadic repository modulus	15
7.3	Binary conditioning is quadratic	16
8	Sequential computability by tolerant bisection	18
8.1	Why ordinary bisection is insufficient	18
8.2	The algorithm on a computable sequence	18
8.3	Correctness of every branch	19
8.4	Clamping and the totalized inverse	20
8.5	Completion of the main theorem	20
8.6	An abstract inverse-computability principle	20
9	Analytic consequences of the exact modulus	20
9.1	Uniform continuity without any Hölder exponent	20
9.2	Endpoint asymptotics become modulus asymptotics	21
9.3	Derivative blow-up versus computability	22
10	Numerical and exact-rational checks	22
10.1	Endpoint mass bounds	22
10.2	Centered-spline convergence	24
10.3	What the code deliberately does not claim	24
11	Connections to the wider Fabius–Rvachev corpus	24
11.1	Thue–Morse inclusion–exclusion	24
11.2	q -binomial coefficients and accelerated forward oracles	24
11.3	Infinite sinc products and alternative evaluation	24
11.4	Lambert W , Bell polynomials, and Bernoulli data	25
11.5	Legendre and Lagrange representations	25
11.6	Total positivity, unimodality, and interval masses	25
12	Lean formalization status and remaining roadmap	25
12.1	Why the present noncomputable definition is not an obstacle	25
12.2	The completed structural inverse-modulus layer	25
12.3	The remaining computability module	27
12.4	Remaining formalization order	27
12.5	Reusable generic infrastructure	27
13	Further deductions, conjectures, and research directions	28
13.1	A certified practical inverse evaluator	28
13.2	Sharp integer moduli and the log-periodic phase	28
13.3	Base- b atomic distributions	28
13.4	Certified spectral and polynomial representations	29
13.5	Higher-dimensional monotone transport	29
13.6	Branchwise inversion of the signed global extension	29
14	Conclusion	30
A	Pseudocode with explicit error budgets	30
B	Proof-dependency and status audit	31
C	Corpus crosswalk	32

D Reproducing the exact-rational supplement	33
--	-----------

1 Main result and proof architecture

1.1 The theorem in one statement

Throughout the report, $F : \mathbb{R} \rightarrow [0, 1]$ denotes the bounded Fabius function: it is 0 on $(-\infty, 0]$, 1 on $[1, \infty)$, and on $[0, 1]$ it is the strictly increasing C^∞ function satisfying

$$F(x) + F(1 - x) = 1, \quad F'(x) = 2F(2x) \quad (0 \leq x \leq \tfrac{1}{2}). \quad (1.1)$$

Let $G : [0, 1] \rightarrow [0, 1]$ be its inverse, and define the totalized inverse

$$\overline{G}(y) := G(\text{clamp}(y)), \quad \text{clamp}(y) := \min(1, \max(0, y)). \quad (1.2)$$

Thus $\overline{G}(y) = 0$ for $y \leq 0$ and $\overline{G}(y) = 1$ for $y \geq 1$.

Theorem 1.1 (Computability of the inverse Fabius function). *The inverse $G : [0, 1] \rightarrow [0, 1]$, in the subset-domain version of the Grzegorzczuk definition, and its totalization $\overline{G} : \mathbb{R} \rightarrow \mathbb{R}$ are computable real functions. More explicitly:*

1. G and $\overline{G}$ are sequentially computable. From one algorithm that names a computable sequence (y_i) , one can uniformly compute dyadic approximations to $G(y_i)$ (or $\overline{G}(y_i)$) of any requested precision.
2. For $r \geq 0$, put

$$\Delta_r = \frac{1}{2^{r(r+1)/2}(r+1)^{r+1}}. \quad (1.3)$$

Then, for all $u, v \in \mathbb{R}$,

$$|u - v| < \Delta_r \implies |\overline{G}(u) - \overline{G}(v)| < 2^{-r}. \quad (1.4)$$

3. For $n \geq 1$, let

$$r(n) := \min \{r \geq 1 : 2^r > n\}, \quad (1.5)$$

and define

$$d(n) := 2^{r(n)(r(n)+1)/2}(r(n)+1)^{r(n)+1}, \quad d(0) := 1. \quad (1.6)$$

The integer function d is recursive (indeed primitive recursive after a standard bounded-search coding), and

$$|u - v| < \frac{1}{d(n)} \implies |\overline{G}(u) - \overline{G}(v)| < \frac{1}{n} \quad (n \geq 1). \quad (1.7)$$

The theorem is stronger than a nonconstructive assertion that a continuous inverse on a compact interval is uniformly continuous. It gives a concrete integer modulus, a finite algorithm on names, and a sharp structural identity for the best possible modulus.

1.2 Four proof layers

The argument is organized so that every hidden effectiveness issue is exposed.

1. **Forward oracle.** A centered finite convolution has an exact Thue–Morse truncated-power formula. At rational inputs it is rational, and its error from F is at most 2^{-N} .
2. **Effective injectivity.** Symmetric unimodality shows that an interval of geometric length h has probability mass at least $F(h)$. A direct event gives the computable lower bound $F(2^{-r}) \geq \Delta_r$.
3. **Uniform continuity.** Effective injectivity of F becomes an effective modulus for G . In fact the ordinary modulus of G is exactly G itself.

4. **Name transformation.** Tolerant bisection combines the forward oracle and the inverse modulus. A midpoint is used for a left/right update only when the sign is certified; otherwise it is already a certified output.

The fourth layer matters. Naive bisection that waits until it decides whether $F(c) < y$, $F(c) = y$, or $F(c) > y$ is not an algorithm on arbitrary computable reals: equality of computable reals is undecidable in general. The tolerance branch is what removes that obstruction.

2 Corpus audit, conventions, and nonduplication boundary

2.1 Pinned repository snapshot

The recursive audit was pinned to commit `40fdea4cc0a728189f357389e3f114a2cb00e561` [8]. The GitHub code index at its parent snapshot enumerated 94 files with extension `.tex` below `Analysis/FabiusFunction/docs`; the final merge added two further LaTeX sources, for a total of 96 in the pinned tree. All source paths were inventoried. Primary and consolidated mathematical reports were read directly; generated table fragments and frozen variants were checked together with the parent documents that include them.

The inspected corpus spans the following overlapping families:

- the two source papers and the main Fabius/Rvachev exposition;
- the Lean walkthrough, paper-coverage map, and mathematical glossary;
- exact dyadic arithmetic, Thue–Morse and q -binomial formulae;
- finite convolutions, infinite sinc products, Fourier decay, canonical products, and Poisson/Legendre representations;
- endpoint asymptotics, lower-Lambert phases, Gamma–zeta periodic terms, Bell-polynomial inverse transfer, and all-orders inverse expansions;
- inverse-dyadic sampling, q -Richardson acceleration, Lagrange and Legendre synthesis, total positivity, spectra, and operator viewpoints;
- the final added spectral/ q -Pochhammer and digital-geometry material.

The audit was thematic rather than a search for one spelling. In particular, it traced the bounded CDF, signed global extension, Rvachev function, totalized inverse, finite spline evaluator, exact dyadic evaluator, inverse asymptotics, and the repository’s definitions of sequential computability and effective uniform continuity.

2.2 What the repository already proves

The following inputs are established in the pinned development and are not claimed as new here.

1. The bounded canonical F exists, is unique, strictly increasing on $[0, 1]$, symmetric, C^∞ , and flat at both endpoints.
2. F is the CDF of the random series in [equation \(4.1\)](#) below. Its centered density is Rvachev’s up function, with Fourier transform an infinite dyadic sinc product.
3. Every dyadic value of F is an exact rational and is computed by an executable recurrence. The centered Thue–Morse splines converge uniformly with error at most 2^{-N} .
4. The forward bounded F and the signed global extension are already proved computable real functions. The relevant Lean modules are `FabiusComputability.lean` and `FabiusComput`

ableSpline.lean; they expose the sequential-computability and effective-uniform-continuity clauses, including a primitive-recursive spline evaluator.

5. The module FabiusInverse.lean constructs the clamped totalized inverse fabiusInv, proves both inverse identities, continuity, monotonicity, interior C^∞ regularity, derivative and curvature laws, and endpoint steepness.
6. The inverse/sampling reports derive exact finite-prefix quantile germs, q -binomial Richardson filters, and all-orders endpoint asymptotics for G , including the nonconstant log-periodic Gamma-zeta fluctuation.

2.3 The gap closed here

No theorem named or formulated as sequential computability, effective uniform continuity, or computable-real-function status for fabiusInv occurs in the pinned snapshot. The missing bridge is not a formal triviality. A computable strictly increasing function can be inverted effectively on a compact interval only after one supplies effective injectivity information; ordinary strict monotonicity is qualitative.

This report supplies that information in two forms:

$$F(x + h) - F(x) \geq F(h), \quad (2.1)$$

$$F(2^{-r}) \geq \Delta_r. \quad (2.2)$$

The first is exact and geometric; the second is elementary and recursive. Together with the existing forward evaluator they close both clauses of the requested definition.

The historical statement above remains about the pinned snapshot. In the current post-publication development, InverseModulus.lean formally proves the first inequality and its exact transfer through the clamped inverse, including sharpness and attainment. The second inequality, its recursive packaging, and the tolerant-bisection realizer remain on the human-proof side of the boundary recorded in [section 12](#).

2.4 Three functions that must not be conflated

The documentation repeatedly warns about a notation trap. The bounded CDF F is clamped outside $[0, 1]$. The signed global extension, often denoted $\mathcal{F}$, agrees with F on $[0, 1]$ but oscillates outside it and obeys global dilation/translation identities. Rvachev's up is an even compactly supported bump on $[-1, 1]$. On its support,

$$\text{up}(t) = \begin{cases} F(t + 1), & -1 \leq t \leq 0, \\ F(1 - t), & 0 \leq t \leq 1. \end{cases} \quad (2.3)$$

The inverse in this report is the inverse of the bounded transition $F : [0, 1] \rightarrow [0, 1]$, with the clamped totalization (1.2). It is not an inverse of the signed global extension.

3 Computable-analysis definitions

3.1 Fast names and computable sequences

Following standard computable analysis [1, 2, 3], a convenient representation of a real number x is a fast dyadic Cauchy name: an algorithm returns a dyadic rational q_p such that

$$|x - q_p| \leq 2^{-p}. \quad (3.1)$$

A real sequence $(x_i)_{i \in \mathbb{N}}$ is computable when one algorithm returns $q_{i,p}$ satisfying (3.1) uniformly in both i and p . The repository uses an equivalent signed-natural code for the dyadic numerator, avoiding an opaque integer encoding.

Definition 3.1 (Sequential computability). A function f is *sequentially computable* if every computable real sequence (x_i) in its domain is mapped to a computable sequence $(f(x_i))$, uniformly in the sequence index and requested precision.

The word “uniformly” is implicit in any useful implementation of this definition: the output algorithm receives the input naming algorithm as a subroutine. The construction in [section 8](#) gives such a uniform transformer, and therefore proves the weaker extensional statement as well.

3.2 The reciprocal uniform-continuity convention

Definition 3.2 (Effective uniform continuity). A function $f : \mathbb{R} \rightarrow \mathbb{R}$ is *effectively uniformly continuous* if there is a recursive $d : \mathbb{N} \rightarrow \mathbb{N}$ with $d(n) > 0$ for $n > 0$ such that

$$|x - y| < \frac{1}{d(n)} \implies |f(x) - f(y)| < \frac{1}{n} \quad (n > 0). \quad (3.2)$$

This is the reciprocal convention used in the cited Wikipedia article [9]. The repository implements it in `FabiusComputability.lean`. It is equivalent to the more common binary modulus convention

$$|x - y| < 2^{-m(p)} \implies |f(x) - f(y)| < 2^{-p}, \quad (3.3)$$

provided m is recursive. We give both forms because the binary version reveals the endpoint conditioning much more clearly.

Definition 3.3 (Computable real function). In the Grzegorzczuk formulation used here, a real function is computable if it is sequentially computable and effectively uniformly continuous.

For G defined only on $[0, 1]$, the same definitions are restricted to sequences and pairs of points in that computable compact domain. The totalized $\overline{G}$ lets one use the literal type $\mathbb{R} \rightarrow \mathbb{R}$ without changing the mathematics.

4 Probability, Fourier structure, and the forward function

4.1 The random-series model

The classical probability model may be written as follows [4, 6, 8]. Let $U_1, U_2, \dots$ be independent uniform random variables on $[0, 1]$ and put

$$X := \sum_{k=1}^{\infty} 2^{-k} U_k. \quad (4.1)$$

The series converges absolutely, $0 \leq X \leq 1$, and

$$F(x) = \mathbb{P}[X \leq x]. \quad (4.2)$$

Reflection $U_k \mapsto 1 - U_k$ sends X to $1 - X$ and proves the symmetry in (1.1).

The centered random variable

$$Y := 2X - 1 = \sum_{k=1}^{\infty} 2^{-k} (2U_k - 1) \quad (4.3)$$

has density up. With $\text{sinc } z = \sin z / z$,

$$\mathbb{E} e^{itY} = \prod_{k=1}^{\infty} \text{sinc} \left(\frac{t}{2^k} \right). \quad (4.4)$$

Thus the probability model, the Rvachev bump, and the infinite sinc product are three representations of the same object. The computability proof will use the probability representation in physical space; [section 11](#) explains how the other representations can serve as alternative evaluators.

4.2 Finite sums and their exact CDFs

For $N \geq 1$, define

$$X_N := \sum_{k=1}^N 2^{-k} U_k, \quad R_N := X - X_N. \quad (4.5)$$

Then

$$0 \leq R_N \leq 2^{-N}. \quad (4.6)$$

Let $F_N(x) = \mathbb{P}[X_N \leq x]$. The general inclusion–exclusion formula for a sum of independent, nonidentically scaled uniforms becomes especially rigid for dyadic scales.

Proposition 4.1 (Finite Thue–Morse CDF). *For $N \geq 1$ and $x \in \mathbb{R}$,*

$$F_N(x) = \frac{2^{N(N+1)/2}}{N!} \sum_{j=0}^{2^N-1} (-1)^{s_2(j)} \left(x - \frac{j}{2^N} \right)_+^N, \quad (4.7)$$

where $s_2(j)$ is the sum of the binary digits of j .

Proof. For positive lengths $a_1, \dots, a_N$, the CDF of $\sum a_k U_k$ is

$$\frac{1}{N! \prod_{k=1}^N a_k} \sum_{A \subseteq \{1, \dots, N\}} (-1)^{|A|} \left(x - \sum_{k \in A} a_k \right)_+^N.$$

This follows by integrating the box indicator over the half-space $\sum a_k u_k \leq x$, or inductively by finite differences of truncated powers. Here $a_k = 2^{-k}$ and $\prod a_k = 2^{-N(N+1)/2}$. The map

$$A \mapsto j(A) := \sum_{k \in A} 2^{N-k}$$

is a bijection from subsets to $\{0, \dots, 2^N - 1\}$, with $\sum_{k \in A} 2^{-k} = j(A)/2^N$ and $|A| \equiv s_2(j(A)) \pmod{2}$. Substitution gives (4.7). $\square$

The alternating signs are exactly the Thue–Morse signs. No limiting argument is hidden in (4.7): at rational x it is an exact rational obtained by finitely many integer operations.

4.3 Centered finite splines and a uniform certificate

The deterministic midpoint of the omitted tail is 2^{-N-1} . Define

$$Q_N := X_N + 2^{-N-1}, \quad S_N(x) := \mathbb{P}[Q_N \leq x] = F_N(x - 2^{-N-1}). \quad (4.8)$$

Then [proposition 4.1](#) gives

$$S_N(x) = \frac{2^{N(N+1)/2}}{N!} \sum_{j=0}^{2^N-1} (-1)^{s_2(j)} \left(x - \frac{2j+1}{2^{N+1}} \right)_+^N. \quad (4.9)$$

This is the centered uniform spline used by the repository’s primitive- recursive evaluator.

Lemma 4.2 (Uniform density bound). *The finite CDFs F_N and S_N are globally 2-Lipschitz.*

Proof. The density of $2^{-1}U_1$ is $2\mathbf{1}_{[0,1/2]}$. Convolution with any probability measure cannot increase the L^∞ norm of a density. Hence the density of X_N is bounded by 2. Translating by 2^{-N-1} does not change this bound. Integrating the density over an interval proves the Lipschitz estimate. $\square$

Theorem 4.3 (Certified centered approximation). *For every $N \geq 1$ and every real x ,*

$$|S_N(x) - F(x)| \leq 2^{-N}. \quad (4.10)$$

Proof. Write

$$X = Q_N + E_N, \quad E_N = R_N - 2^{-N-1}.$$

By (4.6), $|E_N| \leq 2^{-N-1}$. Therefore

$$S_N(x - 2^{-N-1}) \leq F(x) \leq S_N(x + 2^{-N-1}).$$

By lemma 4.2, moving the argument by 2^{-N-1} changes S_N by at most $2 \cdot 2^{-N-1} = 2^{-N}$. This gives both sides of (4.10). $\square$

Corollary 4.4 (Self-contained computability of F). *There is an explicit uniform algorithm which, given a rational x and a precision p , returns a rational q with $|q - F(x)| \leq 2^{-p}$. One may take $q = S_N(x)$ with $N \geq p$. Consequently F is sequentially computable; together with its global 2-Lipschitz bound, it is a computable real function.*

Proof. Formula (4.9) is a finite rational computation, and theorem 4.3 supplies its error certificate. At an arbitrary computable input, first replace the input by a sufficiently accurate dyadic; the 2-Lipschitz bound controls this replacement. The construction is uniform in a sequence index. Effective uniform continuity follows from $|F(x) - F(y)| \leq 2|x - y|$; for example the reciprocal modulus $d(n) = 2n$ works. $\square$

Remark 4.5 (Efficiency versus computability). The direct sum has 2^N terms and is not intended as an efficient evaluator. The repository's exact dyadic recurrence, highest-bit reduction, and primitive-recursive natural-number implementation are substantially better. For the theorem here, only termination and a proved error bound are required.

5 Effective positivity, strict increase, and the density shape

5.1 A closed positive lower bound at dyadic points

The existence of an inverse requires strict increase; effective inversion requires a quantitative version. Both begin with an elementary positive probability event.

Lemma 5.1 (Dyadic endpoint box event). *For every $r \geq 0$,*

$$F(2^{-r}) \geq \Delta_r := 2^{-r(r+1)/2} (r+1)^{-(r+1)}. \quad (5.1)$$

In particular, $F(x) > 0$ for every $x > 0$.

Proof. Put $m = r + 1$. For $1 \leq k \leq m$, impose the independent restrictions

$$U_k \leq a_k := \frac{2^{k-r-1}}{m}. \quad (5.2)$$

The largest a_k is $a_m = 1/m \leq 1$, so these are valid events. On their intersection,

$$2^{-k} U_k \leq \frac{2^{-r-1}}{m} \quad (1 \leq k \leq m).$$

After summation the first m terms contribute at most 2^{-r-1} . The unrestricted tail contributes at most

$$\sum_{k=m+1}^{\infty} 2^{-k} = 2^{-m} = 2^{-r-1}.$$

Thus the event in (5.2) implies $X \leq 2^{-r}$. Its probability is

$$\begin{aligned} \prod_{k=1}^m a_k &= m^{-m} 2^{\sum_{k=1}^m (k-r-1)} \\ &= m^{-m} 2^{m(m+1)/2 - m(r+1)} \\ &= 2^{-r(r+1)/2} (r+1)^{-(r+1)} = \Delta_r. \end{aligned}$$

This proves (5.1). Given $x > 0$, choose r with $2^{-r} \leq x$ and use monotonicity of a CDF. $\square$

The bound is deliberately elementary: it uses only independence, the support of the tail, and integer arithmetic. It is not numerically sharp, but [section 7.3](#) shows that its logarithm has the correct two largest orders for computability purposes.

5.2 Strict monotonicity from the random series

Proposition 5.2 (Strict increase). *For $0 \leq a < b \leq 1$,*

$$F(a) < F(b). \quad (5.3)$$

Hence $F : [0, 1] \rightarrow [0, 1]$ is a homeomorphism and $G = F^{-1}$ exists.

Proof. The density of X_N is strictly positive on the interior of its support $(0, 1 - 2^{-N})$. This follows inductively: a convolution of two densities that are positive on the interiors of compact intervals is positive on the interior of their Minkowski-sum interval, because the convolution integral contains a nonempty open region on which both factors are positive.

Choose N so large that $2^{-N} < b - a$. Then the open interval $(a, b - 2^{-N})$ is nonempty and lies below the right endpoint of the support of X_N . Choose a nonempty open subinterval J of it. The preceding positivity gives $\mathbb{P}[X_N \in J] > 0$. For $X_N \in J$ and every possible $0 \leq R_N \leq 2^{-N}$, one has $a < X_N + R_N < b$. Consequently

$$F(b) - F(a) = \mathbb{P}[a < X \leq b] \geq \mathbb{P}[X_N \in J] > 0.$$

Continuity and the endpoint values $F(0) = 0$, $F(1) = 1$ then give the homeomorphism statement. $\square$

5.3 Symmetric strict unimodality

Let

$$p(x) := F'(x), \quad 0 \leq x \leq 1. \quad (5.4)$$

The repository proves that F is C^∞ , so p is continuous. The functional and reflection equations completely determine its qualitative shape.

Proposition 5.3 (Shape of the Fabius density). *The density p satisfies*

$$p(x) = p(1 - x), \quad (5.5)$$

is strictly increasing on $[0, 1/2]$, and is strictly decreasing on $[1/2, 1]$. Equivalently, $p(x)$ is a strictly decreasing function of $|x - 1/2|$.

Proof. On $[0, 1/2]$, (1.1) gives

$$p(x) = 2F(2x).$$

By [proposition 5.2](#), this is strictly increasing. Differentiating $F(x) + F(1 - x) = 1$ gives $p(x) = p(1 - x)$, which reflects the first-half behavior to the second half. $\square$

This simple observation is the geometric heart of the inverse modulus. It is also a point where Rvachev's function enters directly: by (2.3),

$$p(x) = 2 \operatorname{up}(2x - 1). \quad (5.6)$$

Thus [proposition 5.3](#) is the standard single-peaked shape of the compactly supported atomic bump.

6 The least-mass interval and the exact inverse modulus

6.1 Intervals of a prescribed geometric length

For $0 \leq h \leq 1$ and $0 \leq x \leq 1 - h$, define the fixed-length CDF increment

$$M_h(x) := F(x + h) - F(x). \quad (6.1)$$

If μ is the Fabius probability law with CDF F , then the canonical measure-theoretic interpretation is

$$M_h(x) = \mu((x, x + h]).$$

The law is atomless—indeed it has the continuous density $p = F'$ —so the value is unchanged if either or both endpoints are included. In particular,

$$\mu((x, x + h]) = \mu([x, x + h]) = \mu((x, x + h)) = \mu([x, x + h)).$$

We therefore continue to call $M_h(x)$ an interval mass, while (6.1) fixes the canonical CDF convention.

Theorem 6.1 (Endpoint intervals have least Fabius mass). *For every $0 \leq h \leq 1$,*

$$\min_{0 \leq x \leq 1-h} (F(x + h) - F(x)) = F(h). \quad (6.2)$$

More precisely, when $0 < h < 1$, M_h is increasing on $[0, (1 - h)/2]$, symmetric about $(1 - h)/2$, and decreasing on $[(1 - h)/2, 1 - h]$. Here “increasing” and “decreasing” are understood in the non-strict sense. Its two endpoint values are equal to $F(h)$.

Proof. The cases $h = 0$ and $h = 1$ are immediate. Suppose $0 < h < 1$. Reflection gives

$$\begin{aligned} M_h(1 - h - x) &= F(1 - x) - F(1 - h - x) \\ &= (1 - F(x)) - (1 - F(x + h)) = M_h(x), \end{aligned}$$

so M_h is symmetric about $(1 - h)/2$.

For $0 \leq x \leq (1 - h)/2$, compute

$$\left(x + h - \frac{1}{2}\right)^2 - \left(x - \frac{1}{2}\right)^2 = h(2x + h - 1) \leq 0. \quad (6.3)$$

Thus $x + h$ is at least as close to $1/2$ as x is. By the unimodality in [proposition 5.3](#), $p(x + h) \geq p(x)$, and hence

$$M'_h(x) = p(x + h) - p(x) \geq 0. \quad (6.4)$$

The function M_h is differentiable, so the mean-value theorem makes it nondecreasing on the first half of its domain. The reflection identity then makes it nonincreasing on the second half. Finally,

$$M_h(0) = F(h) - F(0) = F(h),$$

and

$$M_h(1 - h) = F(1) - F(1 - h) = 1 - (1 - F(h)) = F(h).$$

Consequently every admissible increment is at least $F(h)$ and both endpoint increments attain that value. $\square$

Corollary 6.2 (Global increment shape and constrained superadditivity). *For every $h \geq 0$, the globally extended increment M_h is nondecreasing on the half-line*

$$x \leq \frac{1 - h}{2}$$

and nonincreasing on the complementary half-line. Consequently, whenever $a, b \geq 0$ and $a + b \leq 1$,

$$F(a) + F(b) \leq F(a + b). \quad (6.5)$$

Proof. The derivative argument above never used $x \geq 0$: from $x \leq (1 - h)/2$ and $h \geq 0$, either $x + h \leq 1/2$ and the monotonicity of p on the left half gives $p(x) \leq p(x + h)$, or $x + h > 1/2$ and reflection replaces $p(x + h)$ by $p(1 - x - h)$, where

$$x \leq 1 - x - h \leq \frac{1}{2}.$$

Thus $M'_h \geq 0$ on the entire left half-line. Reflection $M_h(1 - h - x) = M_h(x)$ gives the right-half statement. For (6.5), apply [theorem 6.1](#) to the interval from a to $a + b$: its length is b , so

$$F(b) \leq F(a + b) - F(a).$$

Rearranging proves the claim. $\square$

Remark 6.3 (A general principle). The proof used only that a probability density on $[0, 1]$ is symmetric and unimodal. Hence [theorem 6.1](#) holds for every continuous symmetric unimodal distribution, with strictness adjusted when the density has plateaus. The Fabius system is special not because the geometric lemma is exotic, but because $F(h)$ has unusually rich exact arithmetic and endpoint asymptotics.

6.2 The inverse's modulus is the inverse itself

For a bounded function H define its ordinary modulus of continuity by

$$\omega_H(\eta) := \sup \{|H(u) - H(v)| : |u - v| \leq \eta\}. \quad (6.6)$$

For G the variables are restricted to $[0, 1]$; for $\overline{G}$ they range over $\mathbb{R}$.

Theorem 6.4 (Exact modulus of the inverse). *For every $\eta \geq 0$, with the inputs in the definition of ω_G restricted to $[0, 1]$,*

$$\omega_G(\eta) = G(\min(\eta, 1)). \quad (6.7)$$

In particular this is $G(\eta)$ for $0 \leq \eta \leq 1$ and is 1 thereafter. For the totalized inverse and every $\eta \geq 0$,

$$\omega_{\overline{G}}(\eta) = G(\min(\eta, 1)). \quad (6.8)$$

More strongly, for all real u, v ,

$$|\overline{G}(u) - \overline{G}(v)| \leq \overline{G}(|u - v|) = G(\min(|u - v|, 1)). \quad (6.9)$$

The associated ordered-gap and subadditivity laws are

$$\overline{G}(v) - \overline{G}(u) \leq \overline{G}(v - u) \quad (u, v \in \mathbb{R}) \quad (6.10)$$

and

$$\overline{G}(x + y) \leq \overline{G}(x) + \overline{G}(y) \quad (x, y \in \mathbb{R}). \quad (6.11)$$

Proof. First let $c, d \in [0, 1]$. Interchanging them if necessary, assume $c \leq d$, and put

$$a = G(c), \quad b = G(d).$$

Monotonicity of G gives $a \leq b$. The inverse identities and [theorem 6.1](#) give

$$F(b - a) \leq F(b) - F(a) = d - c. \quad (6.12)$$

Both $b - a$ and $d - c$ lie in $[0, 1]$. Applying the increasing inverse and using $G(F(b - a)) = b - a$ yields

$$b - a \leq G(d - c).$$

Removing the temporary ordering assumption gives

$$|G(c) - G(d)| \leq G(|c - d|) \quad (c, d \in [0, 1]). \quad (6.13)$$

Now put $c = \text{clamp}(u)$ and $d = \text{clamp}(v)$. Projection onto $[0, 1]$ is nonexpansive and has diameter one, so

$$|c - d| \leq \min(|u - v|, 1).$$

Combining this with (6.13) proves (6.9). If $|u - v| \leq \eta$, monotonicity then gives the upper bound $G(\min(\eta, 1))$ for both moduli. Let $\theta = \min(\eta, 1)$. The pair $(0, \theta)$ belongs to $[0, 1]^2$, has input separation $\theta \leq \eta$, and has output separation

$$|G(\theta) - G(0)| = G(\theta).$$

Thus the displayed upper bound is attained, not merely approached, proving both exact supremum identities.

For (6.10), if $u \leq v$, remove the absolute value from (6.9); then $v - u = |v - u|$. If $v < u$, the left side is nonpositive by monotonicity, whereas $\overline{G}(v - u) \geq 0$. Finally apply (6.10) with $u = x$ and $v = x + y$:

$$\overline{G}(x + y) - \overline{G}(x) \leq \overline{G}(y),$$

which rearranges to (6.11). □

Corollary 6.5 (Effective-injectivity form). *For $0 \leq h \leq 1$ and $u, v \in \mathbb{R}$,*

$$|u - v| < F(h) \implies |\overline{G}(u) - \overline{G}(v)| < h. \quad (6.14)$$

The closed-threshold and contrapositive forms are

$$|u - v| \leq F(h) \implies |\overline{G}(u) - \overline{G}(v)| \leq h, \quad (6.15)$$

$$h \leq |\overline{G}(u) - \overline{G}(v)| \implies F(h) \leq |u - v|. \quad (6.16)$$

The same statements hold for G on $[0, 1]$.

Proof. The strict hypothesis gives $|u - v| < F(h) \leq 1$, so the saturation in the exact modulus disappears. Strict monotonicity of G on $[0, 1]$ therefore gives

$$|\overline{G}(u) - \overline{G}(v)| \leq G(|u - v|) < G(F(h)) = h.$$

The same chain with non-strict inequalities proves (6.15); its contrapositive is (6.16). □

Theorem 6.6 (Exact strict threshold). *For $0 \leq h \leq 1$ and every real δ ,*

$$[\forall u, v \in \mathbb{R}, |u - v| < \delta \implies |\overline{G}(u) - \overline{G}(v)| < h] \iff \delta \leq F(h). \quad (6.17)$$

Thus $F(h)$ is exactly the largest strict universal input threshold for output error below h .

Proof. If $\delta \leq F(h)$, then $|u - v| < \delta$ implies $|u - v| < F(h)$, and corollary 6.5 applies. Conversely, suppose the universal implication holds but $F(h) < \delta$. Take

$$u = 0, \quad v = F(h).$$

Their input separation is $F(h) < \delta$, whereas the inverse identities give

$$|\overline{G}(0) - \overline{G}(F(h))| = |0 - h| = h,$$

contradicting the asserted strict output bound. Notice that no sign hypothesis on δ is needed: for $\delta \leq 0$ the left implication is vacuous and the right inequality follows from $F(h) \geq 0$. □

7 Effective uniform continuity

7.1 A closed primitive-recursive modulus

Combining the exact inverse modulus with the box event gives the promised fully explicit implication.

Theorem 7.1 (Closed dyadic modulus). *For every $r \geq 0$ and all $u, v \in \mathbb{R}$,*

$$|u - v| < \Delta_r \implies |\overline{G}(u) - \overline{G}(v)| < 2^{-r}, \quad (7.1)$$

where Δ_r is given by (5.1).

Proof. By lemma 5.1, $\Delta_r \leq F(2^{-r})$. Apply corollary 6.5 with $h = 2^{-r}$. $\square$

Theorem 7.2 (Reciprocal modulus required by the definition). *For $n \geq 1$, let $r(n)$ be the least positive integer with $2^{r(n)} > n$ and put*

$$d(n) = 2^{r(n)(r(n)+1)/2} (r(n) + 1)^{r(n)+1}, \quad d(0) = 1. \quad (7.2)$$

Then d is recursive and satisfies (3.2) for $\overline{G}$. Thus $\overline{G}$ and the restricted inverse G are effectively uniformly continuous.

Proof. The function $r(n)$ can be found by bounded integer search, or equivalently by computing the binary length of n ; hence it is recursive. Exponentiation, multiplication, and composition preserve recursiveness, so d is recursive. By construction,

$$\frac{1}{d(n)} = \Delta_{r(n)} \quad \text{and} \quad 2^{-r(n)} < \frac{1}{n}.$$

If $|u - v| < 1/d(n)$, theorem 7.1 gives $|\overline{G}(u) - \overline{G}(v)| < 2^{-r(n)} < 1/n$. $\square$

Remark 7.3 (An even simpler, much larger modulus). Taking $r = n$ gives the one-line primitive-recursive choice

$$d_0(n) = 2^{n(n+1)/2} (n+1)^{n+1} \quad (n \geq 1),$$

because $2^{-n} < 1/n$. The logarithmic choice $r(n)$ in theorem 7.2 is far smaller and has the correct leading complexity scale.

7.2 The exact-dyadic repository modulus

Put $h_n := 2^{-r(n)}$. The corpus proves that every $F(h_n)$ is an exactly computable positive rational. Therefore one can replace the elementary lower bound $\Delta_{r(n)}$ by the exact endpoint mass while retaining an exact dyadic output proxy.

Proposition 7.4 (Repository-native exact-dyadic modulus). *Let*

$$d_*(n) := \left\lceil \frac{1}{F(h_n)} \right\rceil \quad (n \geq 1), \quad h_n = 2^{-r(n)}, \quad d_*(0) := 1. \quad (7.3)$$

Then d_ is recursive and*

$$|u - v| < \frac{1}{d_*(n)} \implies |\overline{G}(u) - \overline{G}(v)| < h_n < \frac{1}{n}. \quad (7.4)$$

Moreover, $d_(n)$ is the least positive integer d satisfying*

$$\frac{1}{d} \leq F(h_n).$$

Equivalently, it is the least reciprocal denominator certified by the exact inverse-modulus law for the fixed dyadic output target h_n . It is not claimed to be the least integer modulus for the weaker target $1/n$.

Proof. The exact dyadic evaluator computes the positive rational $F(h_n)$, and rational reciprocal and ceiling are computable integer operations. By the defining property of the ceiling,

$$\frac{1}{d_*(n)} \leq F(h_n).$$

The strict input hypothesis and [corollary 6.5](#) therefore give output separation strictly below h_n , while the definition of $r(n)$ gives $h_n < 1/n$.

For the qualified minimality statement, let $1 \leq d < d_*(n)$. Then

$$d < \frac{1}{F(h_n)}, \quad \text{hence} \quad \frac{1}{d} > F(h_n).$$

The endpoint pair $u = 0, v = F(h_n)$ has input separation strictly below $1/d$ and output separation exactly h_n . Thus d cannot certify the stricter conclusion $|\overline{G}(u) - \overline{G}(v)| < h_n$. $\square$

Remark 7.5 (The genuine integer optimum for the $1/n$ target). The exact inverse-modulus law shows that a positive integer d has the universal property

$$|u - v| < \frac{1}{d} \implies |\overline{G}(u) - \overline{G}(v)| < \frac{1}{n}$$

if and only if

$$\frac{1}{d} \leq F(1/n).$$

Consequently the mathematically least such denominator is

$$d_{\text{opt}}(n) = \left\lceil \frac{1}{F(1/n)} \right\rceil.$$

The present report does not prove that $n \mapsto d_{\text{opt}}(n)$ is recursive: computability of the real number $F(1/n)$ alone does not make the discontinuous ceiling operation effective at an unknown integer boundary. The dyadic quantity d_* avoids that issue because $F(h_n)$ is an exact rational.

The distinction is already visible at $n = 1$. Here $r(1) = 1$, $F(1/2) = 1/2$, and hence $d_*(1) = 2$, whereas $d = 1$ already satisfies the required implication with output tolerance 1.

7.3 Binary conditioning is quadratic

The exact modulus permits a sharp, representation-independent measurement of how many input bits are intrinsically needed near the flat endpoint.

Definition 7.6 (Optimal binary modulus index). For $p \geq 1$, define

$$\mu(p) := \min \{m \in \mathbb{N} : 2^{-m} \leq F(2^{-p})\} = \left\lceil \log_2 \frac{1}{F(2^{-p})} \right\rceil. \quad (7.5)$$

Then

$$|u - v| < 2^{-\mu(p)} \implies |\overline{G}(u) - \overline{G}(v)| < 2^{-p}, \quad (7.6)$$

and no smaller m has this universal property.

The lower event bound already gives an upper bound for $\mu(p)$. A matching lower bound follows from endpoint flatness.

Lemma 7.7 (Elementary flatness upper bound). For every integer $q \geq 1$ and $0 \leq x \leq 2^{-q}$,

$$F(x) \leq \frac{2^{q(q+1)/2}}{q!} x^q. \quad (7.7)$$

In particular,

$$F(2^{-p}) \leq \frac{2^{-p(p-1)/2}}{p!}. \quad (7.8)$$

Proof. Repeated differentiation of $F'(x) = 2F(2x)$ gives

$$F^{(q)}(x) = 2^{q(q+1)/2} F(2^q x) \quad (0 \leq x \leq 2^{-q}). \quad (7.9)$$

All derivatives of F vanish at 0, and $0 \leq F \leq 1$. Taylor's formula in integral form therefore gives

$$\begin{aligned} F(x) &= \frac{1}{(q-1)!} \int_0^x (x-t)^{q-1} F^{(q)}(t) dt \\ &\leq \frac{2^{q(q+1)/2}}{(q-1)!} \int_0^x (x-t)^{q-1} dt = \frac{2^{q(q+1)/2}}{q!} x^q. \end{aligned}$$

Set $q = p$ and $x = 2^{-p}$ to obtain (7.8). $\square$

Theorem 7.8 (Asymptotically sharp input-bit requirement). *For every $p \geq 1$,*

$$\left\lceil \frac{p(p-1)}{2} + \log_2(p!) \right\rceil \leq \mu(p) \quad (7.10)$$

$$\leq \left\lceil \frac{p(p+1)}{2} + (p+1) \log_2(p+1) \right\rceil. \quad (7.11)$$

Consequently

$$\boxed{\mu(p) = \frac{p^2}{2} + p \log_2 p + \mathcal{O}(p).} \quad (7.12)$$

Proof. The flatness bound (7.8) yields

$$\log_2 \frac{1}{F(2^{-p})} \geq \frac{p(p-1)}{2} + \log_2(p!),$$

while lemma 5.1 yields

$$\log_2 \frac{1}{F(2^{-p})} \leq \frac{p(p+1)}{2} + (p+1) \log_2(p+1).$$

Take ceilings and use (7.5). Stirling's formula $\log_2(p!) = p \log_2 p - (\log_2 e)p + \mathcal{O}(\log p)$ makes both bounds equal to $p^2/2 + p \log_2 p + \mathcal{O}(p)$. $\square$

Remark 7.9 (Intrinsic endpoint cost). The quadratic precision loss is not an artifact of the spline evaluator or of bisection. The two admissible inputs 0 and $F(2^{-p})$ have outputs separated by 2^{-p} . Any uniform oracle algorithm that promises p output bits must, in the worst endpoint case, obtain enough input information to distinguish those values. The exact modulus quantifies that information barrier.

Corollary 7.10 (Growth of the reciprocal modulus). *For $n \rightarrow \infty$, an asymptotically scale-optimal reciprocal modulus has*

$$\log_2 d(n) = \frac{1}{2}(\log_2 n)^2 + (\log_2 n) \log_2 \log_2 n + \mathcal{O}(\log n). \quad (7.13)$$

Equivalently,

$$d(n) = n^{\frac{1}{2} \log_2 n + \log_2 \log_2 n + \mathcal{O}(1)}. \quad (7.14)$$

Proof. Use $p = r(n) = \log_2 n + \mathcal{O}(1)$ in theorem 7.8. The elementary modulus of theorem 7.2 obeys the same two leading orders directly. $\square$

8 Sequential computability by tolerant bisection

8.1 Why ordinary bisection is insufficient

Suppose y is given only by a computable name. At a dyadic midpoint c , one can approximate both $F(c)$ and y , but in general no algorithm can decide whether the two represented reals are exactly equal. An implementation that loops until it proves either $F(c) < y$ or $F(c) > y$ can therefore fail to terminate when $y = F(c)$.

The inverse modulus provides the missing third branch. If the approximations are too close to certify a sign, then $F(c)$ is close to y ; effective injectivity says that c is close to $G(y)$, so the algorithm may return it.

8.2 The algorithm on a computable sequence

Let (y_i) be a computable sequence in $[0, 1]$. Fix a naming algorithm $q(i, s) \in \mathbb{Q}$ with

$$|q(i, s) - y_i| \leq 2^{-s}. \quad (8.1)$$

We describe an algorithm which, from (i, p) , returns a dyadic $z(i, p)$ with

$$|z(i, p) - G(y_i)| < 2^{-p}. \quad (8.2)$$

Algorithm 8.1 (Tolerant certified bisection). Given an index i and output precision p :

1. Set

$$r = p + 2, \quad h = 2^{-r}, \quad \delta = \Delta_r.$$

Choose integers s, N effectively so that

$$2^{-s} \leq \delta/8, \quad 2^{-N} \leq \delta/8. \quad (8.3)$$

Compute the rational $\tilde{y} = q(i, s)$.

2. Initialize the certified bracket $[a, b] = [0, 1]$.

3. Repeat at most $p + 2$ times:

- (a) Put $c = (a + b)/2$ and evaluate the exact rational $S_N(c)$ by (4.9).
- (b) Form the rational approximate difference

$$\tilde{D} = S_N(c) - \tilde{y}. \quad (8.4)$$

- (c) If $\tilde{D} > \delta/2$, replace b by c .
- (d) If $\tilde{D} < -\delta/2$, replace a by c .
- (e) Otherwise return c .

4. If no midpoint was returned, return $(a + b)/2$ after the final update.

Every comparison in [algorithm 8.1](#) is between exact rationals. The order N can be extremely large, but it is obtained by a finite integer calculation and the spline sum is finite. Hence the procedure is an algorithm in the computability-theoretic sense.

8.3 Correctness of every branch

Lemma 8.2 (Difference certificate). *At every midpoint c in [algorithm 8.1](#),*

$$\left| \tilde{D} - (F(c) - y_i) \right| \leq \delta/4. \quad (8.5)$$

Proof. By [theorem 4.3](#) and (8.3), $|S_N(c) - F(c)| \leq \delta/8$. By (8.1) and the same choice of s , $|\tilde{y} - y_i| \leq \delta/8$. The triangle inequality gives (8.5). $\square$

Lemma 8.3 (Safe left/right updates). *If $\tilde{D} > \delta/2$, then $G(y_i) < c$. If $\tilde{D} < -\delta/2$, then $c < G(y_i)$.*

Proof. In the first case, [lemma 8.2](#) gives

$$F(c) - y_i \geq \tilde{D} - \delta/4 > \delta/4 > 0.$$

Since F is strictly increasing, $c > G(y_i)$. The second case is symmetric. Thus every update preserves the invariant

$$G(y_i) \in [a, b]. \quad (8.6)$$

$\square$

Lemma 8.4 (The inconclusive branch is a success branch). *If $|\tilde{D}| \leq \delta/2$, then*

$$|c - G(y_i)| < h = 2^{-r} < 2^{-p}. \quad (8.7)$$

Proof. By [lemma 8.2](#),

$$|F(c) - y_i| \leq |\tilde{D}| + \delta/4 \leq 3\delta/4 < \delta.$$

The choice $\delta = \Delta_r$ and [theorem 7.1](#), applied to the input pair $F(c), y_i$, give

$$|G(F(c)) - G(y_i)| < 2^{-r}.$$

Because $c \in [0, 1]$, $G(F(c)) = c$. Finally, $r = p + 2$. $\square$

Theorem 8.5 (Correctness and uniformity of tolerant bisection). *Algorithm 8.1 terminates and returns a dyadic $z(i, p)$ satisfying (8.2). The map $(i, p) \mapsto z(i, p)$ is computable whenever the name $(i, s) \mapsto q(i, s)$ is computable.*

Proof. The loop has a fixed finite bound. If it returns through the inconclusive branch, [lemma 8.4](#) proves the error estimate. Otherwise [lemma 8.3](#) preserves the bracket invariant through $p + 2$ bisections. Its final width is $2^{-(p+2)}$, so the final midpoint is at distance at most $2^{-(p+3)} < 2^{-p}$ from $G(y_i)$.

All quantities r, h, δ, s, N are computable from p . Midpoints are dyadic; $S_N(c)$ is an exact finite rational expression; and every branch is a rational comparison. The only access to the sequence is the single call to its naming algorithm. Therefore the construction is uniform in i and p . $\square$

Corollary 8.6 (Sequential computability on the unit interval). *The inverse $G : [0, 1] \rightarrow [0, 1]$ is sequentially computable.*

Proof. Given any computable sequence (y_i) in $[0, 1]$, the output of [algorithm 8.1](#) is a computable fast dyadic name for the sequence $(G(y_i))$. $\square$

8.4 Clamping and the totalized inverse

Lemma 8.7 (Computability of clamping). *If (y_i) is a computable real sequence, then $(\text{clamp}(y_i))$ is a computable sequence.*

Proof. Clamping is 1-Lipschitz. If $q(i, p)$ approximates y_i within 2^{-p} , then the exactly rational quantity $\text{clamp}(q(i, p))$ approximates $\text{clamp}(y_i)$ within the same error. The rational min/max operations are decidable and computable. $\square$

Corollary 8.8 (Sequential computability of the totalized inverse). *The totalized inverse $\overline{G} : \mathbb{R} \rightarrow \mathbb{R}$ is sequentially computable.*

Proof. Given a computable sequence (y_i) , first compute the clamped sequence by [lemma 8.7](#), then apply [corollary 8.6](#). By (1.2), the result is $(\overline{G}(y_i))$. $\square$

8.5 Completion of the main theorem

Proof of theorem 1.1. Sequential computability is [corollaries 8.6](#) and [8.8](#). Effective uniform continuity with the explicit recursive modulus is [theorem 7.2](#). These are exactly the two clauses of [definition 3.3](#). $\square$

8.6 An abstract inverse-computability principle

The Fabius proof illustrates a reusable theorem. Stating it clarifies which properties are genuinely needed and which belong only to this example.

Theorem 8.9 (Effective inversion on a computable compact interval). *Let $f : [0, 1] \rightarrow [0, 1]$ be a strictly increasing homeomorphism. Suppose:*

1. $f(c)$ can be approximated uniformly and effectively at every dyadic rational c ;
2. there is a computable positive rational sequence (α_p) such that for all $0 \leq x \leq 1 - 2^{-p}$,

$$f(x + 2^{-p}) - f(x) \geq \alpha_p. \quad (8.8)$$

Then f^{-1} is sequentially computable and effectively uniformly continuous. A tolerant-bisection algorithm is obtained by replacing Δ_r with α_r in [algorithm 8.1](#).

Proof. Condition (8.8) implies $|u - v| < \alpha_p \Rightarrow |f^{-1}(u) - f^{-1}(v)| < 2^{-p}$ by the same contrapositive argument as [corollary 6.5](#). This gives a binary recursive modulus. The proof of [theorem 8.5](#) uses only this implication, dyadic evaluation of f , and strict monotonicity, so it carries over verbatim. $\square$

For the Fabius function, [theorem 6.1](#) identifies the best possible choice $\alpha_p = F(2^{-p})$, while [lemma 5.1](#) supplies a closed lower substitute requiring no invocation of the exact dyadic evaluator.

9 Analytic consequences of the exact modulus

9.1 Uniform continuity without any Hölder exponent

Effective uniform continuity is sometimes mentally associated with a power-law modulus. The inverse Fabius function is a clean counterexample to that intuition.

Theorem 9.1 (No positive Hölder exponent at the endpoints). *For every $\alpha > 0$ and every $C > 0$, there are arbitrarily small $y > 0$ such that*

$$G(y) > Cy^\alpha. \quad (9.1)$$

By reflection, there are arbitrarily small $1 - y > 0$ such that

$$1 - G(y) > C(1 - y)^\alpha. \quad (9.2)$$

Thus neither G nor $\overline{G}$ is Hölder continuous of any positive order at 0 or 1, despite being effectively uniformly continuous.

Proof. Choose an integer q with $q\alpha > 1$. By lemma 7.7, for all sufficiently small $x > 0$, $F(x) \leq C_q x^q$ with a fixed C_q . If $G(y) \leq C y^\alpha$ held near zero, substitution $y = F(x)$ would give

$$x = G(F(x)) \leq C F(x)^\alpha \leq C C_q^\alpha x^{q\alpha}.$$

After division by x , the right side tends to zero, a contradiction. The right endpoint follows from $G(1 - y) = 1 - G(y)$. $\square$

Corollary 9.2 (The exact modulus is sub-Hölder). *For every $\alpha > 0$,*

$$\frac{\omega_G(\eta)}{\eta^\alpha} = \frac{G(\eta)}{\eta^\alpha} \longrightarrow +\infty \quad (\eta \downarrow 0). \quad (9.3)$$

Proof. Choose an integer q with $q\alpha > 1$ and write $x = G(\eta)$, so that $\eta = F(x)$ and $x \downarrow 0$ with $\eta \downarrow 0$. By lemma 7.7, for all sufficiently small x , $F(x) \leq C_q x^q$. Hence

$$\frac{G(\eta)}{\eta^\alpha} = \frac{x}{F(x)^\alpha} \geq C_q^{-\alpha} x^{1-q\alpha} \longrightarrow +\infty.$$

The equality with ω_G is theorem 6.4. $\square$

The endpoint growth used here is also formalized, independently of the new exact-modulus module, in `FabiusInverseAsymptotic.lean`. The declarations `Fabius.rpow_isLittleO_fabiusInv_at_zero_right` and `Fabius.tendsto_fabiusInv_div_rpow_atTop_at_zero_right` give the sub-Hölder divergence, while `Fabius.fabiusInv_not_isBigO_rpow_at_zero_right` and `Fabius.not_exists_fabiusInv_le_const_mul_rpow_near_zero` give the corresponding impossibility of a positive power-law upper bound. These are analytic inputs already present in the repository, not consequences claimed to have been newly proved by `InverseModulus.lean`.

9.2 Endpoint asymptotics become modulus asymptotics

The inverse endpoint reports in the corpus give much more than computability [8]. Let

$$T = \log(1/\eta), \quad L = \log 2, \quad \rho = \sqrt{\frac{2T}{L}}. \quad (9.4)$$

Suppressing the explicitly known first log-periodic correction, their leading equivalent is

$$G(\eta) \sim G_0(\eta) := \frac{\sqrt{T}}{e\sqrt{L}} \exp(-\sqrt{2LT}) \quad (\eta \downarrow 0). \quad (9.5)$$

More precisely, the corpus obtains a relative expansion of the form

$$\frac{G(\eta)}{G_0(\eta)} = 1 + \frac{\frac{1}{2} \log \rho + \kappa_0 - \Psi(\theta)}{\rho} + \mathcal{O}\left(\frac{(\log \rho)^2}{\rho^2}\right), \quad (9.6)$$

where Ψ is a nonconstant smooth one-periodic Gamma-zeta fluctuation and θ is the lower-Lambert phase shifted by an explicit constant.

Corollary 9.3 (Sharp asymptotic modulus). *The expressions (9.5) and (9.6) are, without any further inversion, the leading and first-correction asymptotics of the exact continuity modulus of G and $\bar{G}$:*

$$\omega_G(\eta) = \omega_{\bar{G}}(\eta) = G(\eta) \quad (0 \leq \eta \leq 1). \quad (9.7)$$

Every proved all-orders endpoint expansion for the inverse therefore transfers verbatim to an all-orders modulus-of-continuity expansion.

Proof. This is exactly [theorem 6.4](#). □

This observation closes a conceptual loop in the corpus. The lower-Lambert phase and Bell-polynomial inverse transfer were developed to approximate quantiles. The geometric theorem shows that the same quantiles are the optimal global sensitivity profile of the inverse map.

9.3 Derivative blow-up versus computability

On $0 < y < 1$, the repository proves

$$G'(y) = \frac{1}{F'(G(y))} = \frac{1}{2 \operatorname{up}(2G(y) - 1)} > 0. \quad (9.8)$$

The derivative tends to $+\infty$ at both endpoints. There is therefore no global Lipschitz constant and no endpoint condition number bounded in the ordinary differential sense. Nonetheless the exact nonlinear modulus $G(\eta)$ is computable, and tolerant bisection uses precisely that nonlinear conditioning. This is why derivative-based numerical intuition alone is not a substitute for a computable-analysis proof.

10 Numerical and exact-rational checks

The accompanying script `inverse_fabius_computability_experiments.py` is a reproducibility supplement. The proofs do not depend on it. It uses `fractions.Fraction` for the Thue–Morse splines and asserts every inequality it checks.

10.1 Endpoint mass bounds

Define

$$A_r := \frac{r(r-1)}{2} + \log_2(r!), \quad (10.1)$$

$$B_r := \frac{r(r+1)}{2} + (r+1) \log_2(r+1). \quad (10.2)$$

Then [lemmas 5.1](#) and [7.7](#) say

$$A_r \leq \log_2 \frac{1}{F(2^{-r})} \leq B_r. \quad (10.3)$$

The following values show the quadratic squeezing.

The numerical width in [table 1](#) is not a defect in the computability proof. Its order is only linear in r after the common $p^2/2 + p \log_2 p$ terms are removed. Exact dyadic arithmetic or the sharp Lambert expansion can replace the bracket whenever a tight practical modulus is desired.

r	Δ_r	$B_r = -\log_2 \Delta_r$	flatness upper bound	A_r
1	$1.25000000 \cdot 10^{-1}$	3.000000	1.00000000	0.000000
2	$4.62962963 \cdot 10^{-3}$	7.754888	$2.50000000 \cdot 10^{-1}$	2.000000
3	$6.10351563 \cdot 10^{-5}$	14.000000	$2.08333333 \cdot 10^{-2}$	5.584963
4	$3.12500000 \cdot 10^{-7}$	21.609640	$6.51041667 \cdot 10^{-4}$	10.584963
5	$6.54097611 \cdot 10^{-10}$	30.509775	$8.13802083 \cdot 10^{-6}$	16.906891
6	$5.79006996 \cdot 10^{-13}$	40.651484	$4.23855252 \cdot 10^{-8}$	24.491853
7	$2.22044605 \cdot 10^{-16}$	52.000000	$9.46105473 \cdot 10^{-11}$	33.299208
8	$3.75610368 \cdot 10^{-20}$	64.529325	$9.23931126 \cdot 10^{-14}$	43.299208
9	$2.84217094 \cdot 10^{-24}$	78.219281	$4.01011773 \cdot 10^{-17}$	54.469133
10	$9.72815993 \cdot 10^{-29}$	93.053748	$7.83226120 \cdot 10^{-21}$	66.791061

Table 1: Rigorous lower and upper bounds for $F(2^{-r})$. The two logarithmic columns bracket the optimal binary modulus index.

N	x	$S_N(x)$	$ S_N(x) - F(x) $	certified 2^{-N}
4	1/2	$5.0000000000000000 \cdot 10^{-1}$	0	$6.25 \cdot 10^{-2}$
4	1/4	$6.9010416666666671 \cdot 10^{-2}$	$4.34027778 \cdot 10^{-4}$	$6.25 \cdot 10^{-2}$
4	1/8	$3.2552083333333335 \cdot 10^{-3}$	$2.17013889 \cdot 10^{-4}$	$6.25 \cdot 10^{-2}$
8	1/2	$5.0000000000000000 \cdot 10^{-1}$	0	$3.90625 \cdot 10^{-3}$
8	1/4	$6.9442749023437500 \cdot 10^{-2}$	$1.69542101 \cdot 10^{-6}$	$3.90625 \cdot 10^{-3}$
8	1/8	$3.4713745117187500 \cdot 10^{-3}$	$8.47710503 \cdot 10^{-7}$	$3.90625 \cdot 10^{-3}$
12	1/2	$5.0000000000000000 \cdot 10^{-1}$	0	$2.44140625 \cdot 10^{-4}$
12	1/4	$6.9444437821706131 \cdot 10^{-2}$	$6.62273831 \cdot 10^{-9}$	$2.44140625 \cdot 10^{-4}$
12	1/8	$3.4722189108530679 \cdot 10^{-3}$	$3.31136915 \cdot 10^{-9}$	$2.44140625 \cdot 10^{-4}$

Table 2: Exact centered-spline checks. The global certificate is intentionally conservative; the pointwise dyadic errors are much smaller.

10.2 Centered-spline convergence

The exact values

$$F(1/2) = 1/2, \quad F(1/4) = 5/72, \quad F(1/8) = 1/288 \quad (10.4)$$

provide compact checks of (4.9). The script produces the following exact rational errors, displayed decimally.

For $N = 8$, interval length $h = 1/8$, and the dyadic grid $2^{-8}\mathbb{Z}$, the script also verifies exactly that

$$\min_x (S_8(x + 1/8) - S_8(x)) = S_8(1/8) = \frac{455}{131072},$$

with minimizers $x = 0$ and $x = 7/8$. This finite-convolution experiment mirrors [theorem 6.1](#); the theorem itself is the continuous unimodality argument, not a grid inference.

10.3 What the code deliberately does not claim

The naive rational formula scales exponentially with N . The script uses small orders to check identities and error bounds, not to compete with the repository’s exact dyadic evaluator. Likewise, its short bisection transcript illustrates certified comparisons at a modest scale; sequential computability is proved by the abstract finite algorithm, regardless of practical runtime.

11 Connections to the wider Fabius–Rvachev corpus

11.1 Thue–Morse inclusion–exclusion

The forward oracle is not an unrelated approximation scheme. The binary subset map in [proposition 4.1](#) makes the Thue–Morse sign $(-1)^{s_2(j)}$ the exact inclusion–exclusion coefficient of the finite uniform convolution. Thus the name-transformer for G ultimately rests on the same cancellation mechanism that drives the repository’s exact dyadic formulae and discrete-limit identities.

The centered shift $(2j + 1)/2^{N+1}$ is equally structural: it replaces the unknown tail by its mean and halves the elementary uniform error from the uncentered bound 2^{1-N} to 2^{-N} . In computational terms, this one-bit improvement is free.

11.2 q -binomial coefficients and accelerated forward oracles

The finite Thue–Morse sums admit Gaussian-binomial reorganizations; the arithmetic background is developed in [7, 8]. The inverse/sampling corpus uses $q = 1/4$ Richardson filters on the geometric error scale of centered finite approximants; the current snapshot also adds a quarter-base q -Pochhammer factorization of the Rvachev sinc product. None of these identities is needed for mere computability, because [theorem 4.3](#) already gives an effective rate. They are relevant to complexity: a certified accelerated approximation to $F(c)$ can be dropped into [algorithm 8.1](#) without changing one line of its logical proof, provided the accelerated remainder is enclosed rigorously.

11.3 Infinite sinc products and alternative evaluation

Equation (4.4) gives the Fourier transform of up, in the classical infinite-convolution tradition of [5, 6]. Fourier inversion therefore offers an independent evaluator for F or its density. The corpus develops strong decay estimates, Euler/canonical products, and Poisson-summation formulae. A computability proof based on this route would need an explicit truncation error for both the frequency integral and the infinite product. Once those are supplied, tolerant bisection is again unchanged.

This modularity is useful for cross-certification: finite splines work in physical space with exact rationals; sinc products work in frequency space with analytic tail bounds. Agreement of independently certified enclosures is a stronger implementation check than agreement of unproved decimal approximations.

11.4 Lambert W , Bell polynomials, and Bernoulli data

The flat endpoint makes the inverse poorly conditioned in any power scale. The lower branch W_{-1} naturally solves the saddle equation controlling $F(x)$ as $x \downarrow 0$. The repository then uses Bell-polynomial reversion to transport all-orders forward expansions to $G(y)$ [8]; Bernoulli numbers and polynomials enter through logarithms of the Laplace/sinc products and through formal composition.

The exact identity $\omega_G(\eta) = G(\eta)$ gives these expansions a second interpretation: they are not only quantile asymptotics but optimal continuity- modulus asymptotics. In particular, the nonconstant periodic Gamma-zeta term cannot be discarded from a genuinely sharp effective modulus.

11.5 Legendre and Lagrange representations

The corpus expands up in even Legendre polynomials and represents polynomials by finite combinations of shifted atomic functions. Such bases can provide spectral or interpolation-based forward oracles. For a formal computability theorem, however, convergence alone is insufficient: one needs a recursive tail majorant uniform in the evaluation point. Deriving sharp certified Legendre-tail bounds and comparing their bit complexity with the Thue-Morse spline is a natural next step.

11.6 Total positivity, unimodality, and interval masses

The least-mass theorem used only symmetric unimodality, while several corpus reports investigate stronger total-positivity and variation-diminishing properties. If a stronger kernel theorem is established, it may control not only one interval mass but higher finite differences and multidimensional quantile maps. The computability proof therefore isolates a low-order shadow of a potentially broader shape theory.

12 Lean formalization status and remaining roadmap

12.1 Why the present noncomputable definition is not an obstacle

The repository defines `fabiusInv` by inverting an order isomorphism and then applying `IccExtend`. Lean marks that definition `noncomputable`, because the inverse equivalence is constructed by classical existence machinery. This is a statement about the chosen term in Lean’s programming language, not a theorem that the represented real function has no algorithm.

Computable analysis deliberately separates these two levels. An extensional real function may be introduced by a classical characterization and later be shown to possess a computable *realizer*: a program transforming names of inputs into names of outputs. [Algorithm 8.1](#) is exactly such a realizer. Its correctness is proved by showing that its output denotes the same extensional function as `fabiusInv`.

This distinction already occurs on the forward side. The analytic canonical Fabius function is defined abstractly, whereas `FabiusComputableSpline.lean` supplies a primitive-recursive evaluator and proves that it realizes the same function. The inverse formalization can follow the same architecture.

12.2 The completed structural inverse-modulus layer

The structural part of [section 6](#) now lives in

`Analysis/FabiusFunction/Lean/FabiusFunction/InverseModulus.lean`.

It imports `FabiusInverse.lean` and deliberately remains independent of the heavier random-series and computable-analysis interfaces. Its complete public theorem surface was compiler-checked as one focused target; each theorem uses only the standard logical principles already present in its dependencies. The declaration-level correspondence is as follows.

Report object or claim	Lean declaration
The fixed-length increment $M_h(x) = F(x + h) - F(x)$	<code>Fabius.fabiusIntervalMass.</code>
Reflection $M_h(1 - h - x) = M_h(x)$	<code>Fabius.fabiusIntervalMass_reflect.</code>
Global left-half monotonicity and right-half antitonicity	<code>Fabius.monotoneOn_fabiusIntervalMass_firstHalf</code> and <code>Fabius.antitoneOn_fabiusIntervalMass_secondHalf</code> . These formal half-line statements strengthen the interval-restricted shape in theorem 6.1 .
Least endpoint increment $F(b - a) \leq F(b) - F(a)$	<code>Fabius.fabiusReal_sub_le_sub.</code>
Constrained forward superadditivity (6.5)	<code>Fabius.fabiusReal_add_le.</code>
Ordered inverse gap on $[0, 1]$	<code>Fabius.fabiusInv_sub_le_sub_of_mem_Icc.</code>
Ordered inverse gap including clamped tails, and its order-free strengthening (6.10)	<code>Fabius.fabiusInv_sub_le_sub_of_le</code> and <code>Fabius.fabiusInv_sub_le_sub.</code>
Global inverse subadditivity (6.11)	<code>Fabius.fabiusInv_add_le.</code>
Absolute and metric pointwise modulus (6.9)	<code>Fabius.abs_fabiusInv_sub_le</code> and <code>Fabius.dist_fabiusInv_le.</code>
The saturation identity $\overline{G}(\min(\eta, 1)) = \overline{G}(\eta)$	<code>Fabius.fabiusInv_min_one.</code>
Attained exact totalized modulus and its supremum identity	<code>Fabius.isGreatest_abs_fabiusInv_sub</code> and <code>Fabius.sSup_abs_fabiusInv_sub_eq.</code>
Attained exact modulus with unit-interval inputs, for every $\eta \geq 0$	<code>Fabius.isGreatest_abs_fabiusInv_sub_Icc</code> and <code>Fabius.sSup_abs_fabiusInv_sub_Icc_eq.</code>
Strict and closed effective injectivity	<code>Fabius.abs_fabiusInv_sub_lt_of_abs_sub_lt_fabiusReal</code> and <code>Fabius.abs_fabiusInv_sub_le_of_abs_sub_le_fabiusReal.</code>
Contrapositive separation (6.16)	<code>Fabius.fabiusReal_le_abs_sub_of_le_abs_fabiusInv_sub.</code>
Exact strict-threshold biconditional (6.17)	<code>Fabius.forall_abs_fabiusInv_sub_lt_iff.</code>

The formal least-mass API is intentionally expressed as an inequality plus explicit endpoint attainment rather than through a separate minimum operator. The `IsGreatest` declarations record both the sharp upper bound and an extremizing pair; the `sSup` equalities are immediate corollaries. The totalized formal theorem returns `fabiusInv ... eta`, which is literally the report's $G(\min(\eta, 1))$ because `fabiusInv` is already clamped.

12.3 The remaining computability module

A natural downstream file remains

Analysis/FabiusFunction/Lean/FabiusFunction/FabiusInverseComputability.lean.

It should import FabiusFunction.InverseModulus. It no longer needs to reprove interval shape or inverse-gap estimates. Its remaining public surface is the effective layer; the following Lean is schematic rather than a claim about declarations already present.

```

/-- Closed elementary dyadic lower bound. -/
theorem delta_le_fabiusAtInverseTwoPow (r : Nat) :
  delta r <= fabiusReal F ((2 : Real) ^ (-r : Int))

/-- The totalized inverse has a recursive reciprocal modulus. -/
theorem fabiusInv_effectivelyUniformContinuous :
  EffectivelyUniformContinuous (fabiusInv fabius fabius_spec)

/-- Tolerant bisection transforms every computable input sequence. -/
theorem fabiusInv_sequentiallyComputable :
  SequentiallyComputable (fabiusInv fabius fabius_spec)

/-- Both Grzegorzczuk clauses, packaged together. -/
theorem fabiusInv_isComputableRealFunction :
  IsComputableRealFunction (fabiusInv fabius fabius_spec)

```

12.4 Remaining formalization order

The former structural roadmap steps are complete in InverseModulus.lean. The remaining order is:

1. Connect the exact dyadic evaluator to the positive rational $F(2^{-r})$ and package the reciprocal-ceiling modulus, with the qualified minimality statement of [proposition 7.4](#).
2. Formalize the elementary box-event bound $\Delta_r \leq F(2^{-r})$ as a portable closed alternative.
3. Formalize a generic tolerant-bisection lemma for a monotone computable map equipped with a recursive lower-increment bound. The result should not be Fabius-specific.
4. Instantiate the generic lemma with the primitive-recursive centered spline evaluator from FabiusComputableSpline.lean.
5. Package sequential computability and effective uniform continuity into the repository's computable-real-function interface.
6. Only after the effective core is complete, formalize the flatness bounds and input-bit asymptotics; they are not logical prerequisites of the computability theorem.

12.5 Reusable generic infrastructure

The abstract [theorem 8.9](#) deserves its own module. In a representation-oriented library it can be stated using an oracle for f and a recursive lower-separation function

$$\alpha(p) > 0, \quad x + 2^{-p} \leq b \implies f(x + 2^{-p}) - f(x) \geq \alpha(p).$$

The conclusion is a computable realizer for the inverse. Tolerant bisection then becomes library infrastructure usable for distribution quantiles, implicit monotone roots, cumulative spectral measures, and monotone solutions of integral equations. The proof is constructive in the strong sense that all precision budgets are explicit.

13 Further deductions, conjectures, and research directions

13.1 A certified practical inverse evaluator

The proof algorithm is intentionally elementary rather than efficient. A practical certified implementation should retain its logical shell while replacing the forward oracle by a portfolio of enclosures:

- exact dyadic evaluation and terminating local Taylor formulae near dyadic anchors;
- centered Thue–Morse splines at moderate depth;
- quarter-base q -Richardson acceleration of finite-prefix errors;
- Fourier/sinc evaluation in regions where frequency-space decay is favorable;
- lower-Lambert endpoint enclosures when the requested target is extremely close to 0 or 1.

Each backend must return a rational interval containing $F(c)$, not merely a floating-point approximation. The bisection controller only asks whether that interval lies wholly above or below the target interval; overlapping intervals trigger refinement or the modulus-based acceptance branch. Thus backend selection is a performance concern, not part of the correctness argument.

Research problem 13.1 (Near-optimal certified evaluation). *Construct and formally verify an inverse evaluator whose oracle-bit consumption is $\mu(p) + \mathcal{O}(\log p)$ for p output bits at the worst endpoint, and whose arithmetic running time is quasi-linear or softly quadratic in the size of the exact integers produced. Separate information-theoretic oracle precision from the cost of evaluating F to that precision.*

13.2 Sharp integer moduli and the log-periodic phase

The real-valued optimal input-bit threshold is

$$\Lambda(p) := \log_2 \frac{1}{F(2^{-p})}, \quad \mu(p) = \lceil \Lambda(p) \rceil.$$

The all-orders endpoint theory expresses $\Lambda(p)$ in a lower-Lambert phase with a nonconstant one-periodic Gamma–zeta contribution and a hierarchy of inverse powers. Passing to the integer ceiling introduces a second, sawtooth nonlinearity. It is therefore natural to study the arithmetic interaction between the smooth periodic saddle phase and fractional parts of its growing nonperiodic carrier.

Conjecture 13.2 (Nontrivial fluctuation of the optimal integer modulus). *After subtracting any fixed finite truncation of the full nonperiodic asymptotic expansion of $\Lambda(p)$, the bounded residual governing $\mu(p)$ does not become eventually constant or eventually periodic in the integer variable p . Its accumulation set is generated jointly by the Gamma–zeta phase and the ceiling sawtooth.*

The conjecture is deliberately qualitative. A stronger equidistribution claim would require information about fractional parts of a nonlinear Lambert-type phase that is not supplied by the present corpus. Exact values $F(2^{-p}) \in \mathbb{Q}$ make the sequence amenable to arbitrary-precision experimentation without numerical ambiguity.

13.3 Base- b atomic distributions

Replace the dyadic random series by

$$X_b = \sum_{k \geq 1} b^{-k} U_k, \quad b \geq 2,$$

with independent uniform coordinates. The finite convolution is still computable, but binary Thue–Morse signs are replaced by base- b subset or digit combinatorics, and the support is $[0, 1/(b-1)]$

before normalization. The least-interval-mass theorem continues to follow whenever the normalized density is symmetric and unimodal. The same proof would then identify the optimal inverse modulus with the quantile itself.

Research problem 13.3 (Uniform computability across the base). *Develop a theorem uniform in the integer parameter b : construct a single algorithm that, given b , a name of y , and a precision p , computes the normalized quantile of X_b . Determine the leading dependence of the optimal input-bit modulus on both b and p , and identify the corresponding base- b log-periodic correction.*

13.4 Certified spectral and polynomial representations

The Legendre, Lagrange, and shifted-up representations in the corpus are mathematically exact, but a representation becomes an algorithm only after a recursive uniform tail bound is attached. This suggests three concrete projects:

1. derive explicit Legendre coefficient envelopes strong enough to certify pointwise and uniform evaluation of up;
2. combine the finite shifted-up representation of polynomials with interval arithmetic to obtain certified local surrogates for F ;
3. compare physical-space spline, Legendre, and sinc-product evaluators by bit complexity, not merely by observed decimal convergence.

A cross-certified implementation could evaluate the same point by two independent representations and intersect the resulting enclosures. This is particularly attractive near transition regions where one representation's natural error bound is pessimistic.

13.5 Higher-dimensional monotone transport

The one-dimensional theorem

$$\min_x \mathbb{P}\{X \in (x, x + h]\} = \mathbb{P}\{X \in (0, h]\}$$

is a sharp anti-concentration lower bound tailored to a symmetric unimodal law. Because the law is atomless, the same identity holds with any choice of open or closed endpoints. In higher dimensions, inverse CDFs are replaced by triangular Rosenblatt maps or optimal-transport maps. Effective invertibility would require computable lower bounds on masses of suitable boxes or sections.

Research problem 13.4 (Atomic transport computability). *For products or coupled variants of Fabius-type laws, determine conditions under which the triangular transport from Lebesgue measure has a computable inverse with an explicit multivariate modulus. Investigate whether total positivity or variation-diminishing properties supply the needed lower section-mass estimates.*

13.6 Branchwise inversion of the signed global extension

The signed global Fabius extension is not globally injective, so it has no single inverse. On every maximal monotonicity interval, however, it has a branch inverse. Local finiteness of its Thue–Morse translate expansion makes forward evaluation computable. The remaining issue is effective branch separation near critical values.

Research problem 13.5 (Computable branch atlas). *Construct an effective enumeration of monotonicity intervals of the signed global extension, together with certified branch ranges and recursive inverse moduli. Determine whether the self-similar translation law permits one finite branch template plus a Thue–Morse address.*

14 Conclusion

The inverse Fabius function is computable in precisely the two-part sense requested. The effective-uniform-continuity clause follows from the exact geometric identity

$$\omega_G(\eta) = G(\eta),$$

combined with the closed positive lower bound

$$F(2^{-r}) \geq 2^{-r(r+1)/2}(r+1)^{-(r+1)}.$$

The sequential-computability clause follows from a tolerant bisection transformer that uses only finite exact rational operations, a computable name of the target, and certified centered-spline evaluations of F . It never requires deciding equality of computable reals.

The result is stronger than an existence proof. It identifies the *optimal* modulus, explains why inversion loses quadratically many bits at the flat endpoints, and turns the repository's lower-Lambert quantile asymptotics into sharp continuity-modulus asymptotics. The same proof covers the inverse on $[0, 1]$ and the canonical clamped totalization on all of $\mathbb{R}$.

Conceptually, the decisive bridge is simple: for a symmetric unimodal density, endpoint intervals have the least mass. In the Fabius setting that shape fact connects probability, computable analysis, exact dyadic arithmetic, and inverse asymptotics. It is also the part most likely to generalize beyond the particular atomic function studied here.

A Pseudocode with explicit error budgets

The following version makes every tolerance in [algorithm 8.1](#) explicit. The function `TargetName(i, s)` returns a rational within 2^{-s} of the clamped input y_i . The function `CenteredSpline(N, c)` is the exact rational $S_N(c)$ of (4.9).

```
InverseFabiusName(i, p):
  # Requested absolute output error: 2^(-p).
  r      := p + 2
  h      := 2^(-r)
  delta  := 2^(-r*(r+1)/2) * (r+1)^(-(r+1))

  choose s with 2^(-s) <= delta/8
  choose N with 2^(-N) <= delta/8
  yhat   := clamp(TargetName(i, s), 0, 1)

  a := 0
  b := 1

  repeat p + 2 times:
    c := (a + b)/2
    fhat := CenteredSpline(N, c)
    D := fhat - yhat

    # |D - (F(c)-y_i)| <= delta/4.
    if D > delta/2:
      b := c                # certified F(c) > y_i
    else if D < -delta/2:
      a := c                # certified F(c) < y_i
    else:
      return c              # |F(c)-y_i| < delta

  return (a + b)/2
```

In the acceptance branch, $|F(c) - y_i| < 3\delta/4 < \delta \leq F(h)$, so [corollary 6.5](#) gives $|c - G(y_i)| < h < 2^{-p}$. If no acceptance occurs, every update is certified and the true inverse remains in $[a, b]$; after $p + 2$ iterations its width is $2^{-(p+2)}$, so the final midpoint is within $2^{-(p+3)} < 2^{-p}$.

The constants have generous slack. They were chosen to make the proof transparent and to absorb representation-dependent rounding. Replacing the three occurrences of $1/8, 1/4, 1/2$ by any fixed compatible margin gives the same computability theorem.

B Proof-dependency and status audit

Claim	Input	Human proof and Lean status
Random-series representation of F	Infinite convolution / probability representation in the repository and classical Fabius theory	Imported; normalization restated in section 4 .
Exact finite CDF (4.7)	Inclusion–exclusion for finite sums of uniforms	Proved in proposition 4.1 .
Uniform centered-spline error 2^{-N}	Tail support and the global density bound $p \leq 2$	Proved in theorem 4.3 ; also formalized forward in the repository.
Strict increase of F	Positivity of finite-convolution densities and a short tail	Proved in proposition 5.2 .
Symmetric strict unimodality of $p = F'$	Differential equation, symmetry, and strict increase of F	Proved in proposition 5.3 from imported Fabius identities.
Least fixed-length increment $F(h)$	Symmetric unimodality of p	Human proof in theorem 6.1 ; reflection, global non-strict shape, and the lower bound are formalized in <code>InverseModulus.lean</code> . Exact names appear in section 12 .
Exact inverse modulus $G(\eta)$	Least interval mass and inverse identities	Human proof in theorem 6.4 ; attained maxima and exact suprema are formalized in <code>InverseModulus.lean</code> . Exact names appear in section 12 .
Exact effective-injectivity threshold $F(h)$	Pointwise inverse modulus and inverse identities	Human proofs in corollary 6.5 and theorem 6.6 ; strict, closed, contrapositive, and biconditional forms are formalized in <code>InverseModulus.lean</code> .
Closed modulus Δ_r	Explicit positive box event in the random series	Proved in theorem 7.1 .

Claim	Input	Human proof and Lean status
Effective uniform continuity	Closed modulus and recursive integer arithmetic	Proved in theorem 7.2 . Its structural pointwise estimate is formalized in <code>InverseModulus.lean</code> ; recursive-modulus packaging is not.
Sequential computability	Forward rational oracle, closed inverse modulus, and tolerant bisection	Proved in theorem 8.5 and corollary 8.8 . No tolerant-bisection or sequential-computability declaration is supplied by <code>InverseModulus.lean</code> .
Optimal bit asymptotic	Closed lower bound plus iterated-flatness upper bound	Proved in theorem 7.8 .
Sharp periodic refinement	Repository’s all-orders endpoint inverse theory	Imported and reinterpreted in corollary 9.3 ; not rederived.

This table is also a minimal trust boundary. The main computability theorem does not rely on the conjectures in [section 13](#), on numerical data, or on any asymptotic expansion.

C Corpus crosswalk

Corpus component	Role in the present report
<code>Fabius_Function_and_Rv_achev_Up.tex</code> and archived primary expositions	Normalization, probability law, up-function relation, exact dyadic arithmetic, finite splines, and endpoint asymptotic context.
<code>fabius_lean_walkthrough.tex</code>	Public theorem map; distinction among the bounded CDF, signed extension, and up-function; forward computability and inverse module architecture.
<code>FabiusFunction_Mathematical_Glossary.tex</code>	Repository-specific Grzegorzczuk definitions and terminology for fast names, sequential computability, and effective uniform continuity.
<code>PAPER_COVERAGE.md</code> and the two Arias de Reyna paper transcriptions	Formal/source crosswalk for probability, Fourier, dyadic, moment, and arithmetic results.
<code>FabiusComputability.lean</code> and <code>FabiusComputableSpline.lean</code>	Existing forward 2-Lipschitz theorem, uniform centered-spline certificate, primitive-recursive evaluator, and computable-real-function packaging.
<code>FabiusInverse.lean</code>	Existing clamped inverse, inverse identities, continuity, interior derivatives, curvature, and endpoint steepness.

Corpus component	Role in the present report
<code>InverseModulus.lean</code>	Post-snapshot formalization of fixed-length Fabius increments, their global unimodal shape, the least endpoint increment, constrained forward superadditivity, local and global inverse-gap inequalities, global inverse subadditivity, absolute and metric pointwise moduli, attained exact unit-interval and totalized suprema, and the exact effective-injectivity threshold. Exact declaration names appear in section 12 .
<code>Inverse_and_Sampling_Frontiers.tex</code> and <code>Inverse_Endpoint_All_Orders.tex</code>	Exact finite-prefix quantiles, q -Richardson filters, Bell-polynomial inversion, and all-orders endpoint quantile asymptotics.
<code>Small_Argument_Asymptotics.tex</code>	Explicit lower-Lambert forward asymptotics and periodic Gamma-zeta coefficient calculus.
q -calculus, sinc-product, Fourier-decay, Legendre/Lagrange, total-positivity, and frontier-compilation sources	Context for alternative certified oracles, shape strengthening, and research directions; not logical prerequisites of the computability proof.

The audit included generated `.tex` fragments and frozen variants, but the report cites parent documents rather than treating generated rows as independent mathematical sources. The repository was pinned because its `main` branch advanced during the audit; all path and count statements refer to commit `40fdea4cc0a728189f357389e3f114a2cb00e561`.

D Reproducing the exact-rational supplement

The archive accompanying this report contains

`inverse_fabius_computability_experiments.py`

and its captured output `numerical_output.txt`. The program uses only Python’s standard library. From the extracted package directory, run

```
python inverse_fabius_computability_experiments.py
```

The program performs all spline evaluations with `fractions.Fraction`. Its principal checks are:

1. the endpoint event lower bound and flatness upper bound in [table 1](#);
2. exact convergence of the centered splines at $1/2$, $1/4$, and $1/8$, using the exact values $1/2$, $5/72$, and $1/288$;
3. a finite-grid analogue of the least-interval-mass theorem;
4. a short certified bisection transcript illustrating all comparison margins.

These computations are regression tests and illustrations. No numerical observation is used as a premise of a theorem.

References

- [1] A. Grzegorzcyk, On the definitions of computable real continuous functions, *Fundamenta Mathematicae* **44** (1957), 61–71.
- [2] K. Weihrauch, *Computable Analysis: An Introduction*, Springer, Berlin, 2000.
- [3] M. B. Pour-El and J. I. Richards, *Computability in Analysis and Physics*, Springer, Berlin, 1989.
- [4] J. Fabius, A probabilistic example of a nowhere analytic C^∞ -function, *Zeitschrift für Wahrscheinlichkeitstheorie und Verwandte Gebiete* **5** (1966), no. 2, 173–174, <https://doi.org/10.1007/BF00536652>.
- [5] B. Jessen and A. Wintner, Distribution functions and the Riemann zeta function, *Transactions of the American Mathematical Society* **38** (1935), no. 1, 48–88, <https://doi.org/10.1090/S0002-9947-1935-1501802-5>.
- [6] J. Arias de Reyna, An infinitely differentiable function with compact support: definition and properties, English translation of the 1982 paper, <https://arxiv.org/abs/1702.05442>.
- [7] J. Arias de Reyna, Arithmetic of the Fabius function, <https://arxiv.org/abs/1702.06487>.
- [8] V. Reshetnikov, ProveIt, Fabius-function development and documentation, repository snapshot 40fdea4cc0a728189f357389e3f114a2cb00e561, <https://github.com/VladimirReshetnikov/ProveIt>.
- [9] Wikipedia contributors, Computable real function, accessed 30 August 2026, https://en.wikipedia.org/wiki/Computable_real_function.